



Модуль 6. Асинхронная работа с базами данных

Введение в модуль

ML-сервис не существует в вакууме. Предсказания нужно где-то хранить, фичи — извлекать, пользователей — аутентифицировать, логи — записывать. Всё это требует постоянного хранилища. В асинхронном мире доступ к базе данных — это I/O-операция, идеально лежащая на event loop, но с собственными сложностями: транзакции, изоляция, пулы соединений, миграции схемы.

В этом модуле мы разберём:

- теорию реляционных баз: ACID, уровни изоляции, реляционную алгебру Кодда;
- SQLAlchemy 2.0 — де-факто стандарт ORM в Python, полностью переработанный для асинхронности;
- интеграцию с FastAPI через Dependency Injection: сессия на запрос, транзакции, обработка deadlock'ов;
- Alembic — инструмент миграций, без которого невозможен командный разработка;
- NoSQL и специализированные хранилища: Redis, MongoDB, векторные базы — когда реляционной модели недостаточно.

6.1. Реляционные БД: теория для практика

6.1.1. ACID: фундамент надёжности

Любая база данных обещает хранить данные. Но что значит «надёжно»? Теория даёт четыре свойства:

Atomicity (Атомарность) Транзакция — это неделимая единица работы. Либо выполняются все её операции, либо ни одной. Если после начала транзакции произойдёт сбой питания, база откатит все частичные изменения.

Пример для ML: логирование предсказания состоит из двух INSERT: в таблицу `predictions` и в таблицу `prediction_features`. Атомарность гарантирует, что не окажется «сиротской» записи в `predictions` без соответствующих фичей.

Consistency (Согласованность) Транзакция переводит базу из одного согласованного состояния в другое. Согласованность определяется **ограничениями (constraints)**: PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE. Если транзакция нарушает ограничение — она откатывается целиком.

Важно: в распределённых системах CAP-теоремы «Consistency» имеет другое значение. Здесь мы говорим о consistency в смысле ACID — целостности данных.

Isolation (Изоляция) Параллельно выполняющиеся транзакции не должны мешать друг другу. Результат параллельного выполнения должен быть эквивалентен результату **какого-то** последовательного выполнения (serializable execution).

На практике полная изоляция дорога. СУБД предлагают **уровни изоляции** — компромисс между корректностью и производительностью.

Durability (Долговечность) После успешного `COMMIT` данные сохранены навсегда, даже если сразу после этого произойдёт сбой. Реализуется через журнал упреждающей записи (WAL, Write-Ahead Log): сначала изменения записываются в журнал на диск, потом применяются к данным.

6.1.2. Уровни изоляции транзакций и феномены

SQL-92 определяет четыре уровня изоляции. Чтобы их понять, нужно знать **феномены** — аномалии, которые могут возникнуть при параллельном доступе.

Феномен 1: Dirty Read («грязное» чтение)

Транзакция А изменяет строку, но ещё не зафиксировала изменения (`COMMIT` не выполнен). Транзакция В читает эту строку. Если А затем откатится (`ROLLBACK`), В прочитала данные, которых «не существует».

In []: Время | Транзакция А | Транзакция В

t1	BEGIN	
t2	UPDATE balance = 1000	
t3		BEGIN
t4		SELECT balance -> 1000 (DIRTY!)
t5	ROLLBACK	
t6		COMMIT

Феномен 2: Non-Repeatable Read (неповторяемое чтение)

Транзакция А читает строку. Транзакция В изменяет эту строку и фиксирует изменения. Транзакция А читает ту же строку снова — получает другой результат.

In []:

Время	Транзакция А	Транзакция В
t1	BEGIN	
t2	SELECT balance -> 500	
t3		BEGIN
t4		UPDATE balance = 600
t5		COMMIT
t6	SELECT balance -> 600	(НЕПОВТОРЯЕМО!)

Феномен 3: Phantom Read (фантомное чтение)

Транзакция А выполняет запрос с условием (WHERE age > 18), получает 10 строк. Транзакция В вставляет новую строку, удовлетворяющую условию, и фиксирует. Транзакция А повторяет тот же запрос — получает 11 строк. «Фантомная» строка появилась из ниоткуда.

In []:

Время	Транзакция А	Транзакция В
t1	BEGIN	
t2	SELECT * WHERE age>18 -> 10	
t3		BEGIN
t4		INSERT user(age=20)
t5		COMMIT
t6	SELECT * WHERE age>18 -> 11	(ФАНТОМ!)

Феномен 4: Lost Update (потерянное обновление)

Две транзакции одновременно читают одну и ту же строку, модифицируют её в памяти и записывают обратно. Одно из изменений перезаписывает другое.

In []:

Время	Транзакция А	Транзакция В
t1	BEGIN	BEGIN

t2		SELECT balance -> 500		SELECT balance -> 500
t3		balance = 500 + 100		balance = 500 + 200
t4		UPDATE balance = 600		
t5		COMMIT		
t6				UPDATE balance = 700
t7				COMMIT

Итог: balance = 700, хотя должно быть 800. Обновление A потеряно.

Уровни изоляции SQL-92

Уровень	Dirty Read	Non-Repeatable	Phantom Read
READ UNCOMMITTED	Возможен	Возможен	Возможен
READ COMMITTED	Невозможен	Возможен	Возможен
REPEATABLE READ	Невозможен	Невозможен	Возможен*
SERIALIZABLE	Невозможен	Невозможен	Невозможен

*В PostgreSQL уровень REPEATABLE READ фактически предотвращает и phantom reads (реализован через MVCC + predicate locking). В MySQL InnoDB — тоже. Но стандарт SQL-92 этого не требует.

Какой уровень выбрать?

- **READ COMMITTED** (по умолчанию в PostgreSQL): хороший баланс. Предотвращает dirty read, но позволяет non-repeatable read. Для большинства веб-приложений достаточно.
- **REPEATABLE READ**: нужен, когда внутри транзакции выполняется несколько связанных запросов, и промежуточные изменения недопустимы. Например, расчёт агрегатов для отчёта.
- **SERIALIZABLE**: самый строгий, но самый медленный. Используется для финансовых операций, где lost update недопустим. В PostgreSQL реализован через SSI (Serializable Snapshot Isolation), что эффективнее, чем блокировка всей таблицы.

6.1.3. Индексы: B-Tree, Hash и стратегии

Индекс — структура данных, ускоряющая выборку за счёт дополнительного хранения и поддержания упорядоченности.

B-Tree (Balanced Tree)

Стандартный индекс в PostgreSQL. B-дерево — это самобалансирующееся дерево поиска с высоким **фактором ветвления**

(тысячи ключей в узле).

Почему не обычное бинарное дерево?

- Дисковые операции дороги (4-10 мс на случайное чтение).
- В-дерево минимизирует высоту дерева: при факторе ветвления 1000 и 1 миллионе записей высота = 2 (корень + листья).
- Узлы соответствуют страницам диска (обычно 8 КБ), что оптимизирует кэширование.

Сложность операций:

- Поиск: $O(\log_B n)$, где B — фактор ветвления.
- Вставка: $O(\log_B n)$ (возможно расщепление узла).
- Удаление: $O(\log_B n)$ (возможно слияние узлов).

Hash-индексы

Хранят хеш-код значения. Поиск по точному равенству: $O(1)$ в среднем. Но:

- Не поддерживают диапазоны (`WHERE age > 18`).
- Не поддерживают сортировку (`ORDER BY`).
- В PostgreSQL 10+ стали crash-safe и рекомендуются реже, чем B-Tree.

Частичные индексы (Partial Indexes)

Индексируют только строки, удовлетворяющие условию:

```
In [ ]: CREATE INDEX idx_active_users ON users(email) WHERE is_active = true;
```

Полезно, когда таблица большая, но запросы работают с небольшим подмножеством (например, активные пользователи).

Покрывающие индексы (Covering Indexes / Index-Only Scan)

Индекс «покрывает» запрос, если все нужные колонки есть в индексе. PostgreSQL может вернуть результат, не обращаясь к таблице (heap):

```
In [ ]: CREATE INDEX idx_users_covering ON users(email) INCLUDE (name, created_at
```

Запрос `SELECT name, created_at FROM users WHERE email = 'a@b.com'` выполнится только по индексу.

6.1.4. Математическая подоплека: теория реляционных баз Кодда и реляционная алгебра

В 1970 году Эдгар Кодд опубликовал статью «A Relational Model of Data for Large Shared Data Banks», положившую начало реляционным СУБД.

Реляционная модель:

- **Домен** — множество значений одного типа (например, `INTEGER`, `VARCHAR`).
- **Отношение (relation)** — подмножество декартова произведения доменов:

$$R \subseteq D_1 \times D_2 \times \dots \times D_n$$

- **Кортеж (tuple)** — элемент отношения, строка таблицы.
- **Атрибут** — колонка, проекция кортежа на один домен.
- **Степень** — число атрибутов (n).
- **Мощность** — число кортежей.

Таблица SQL — это **представление** отношения. В математике отношение — множество, а значит:

- Порядок кортежей не определён (SQL добавляет `ORDER BY`).
- Кортежи уникальны (SQL добавляет `DISTINCT` или `PRIMARY KEY`).
- Порядок атрибутов не важен (в SQL важен при `SELECT *`).

Реляционная алгебра:

Операции, замкнутые относительно множества отношений (результат операции — снова отношение):

Операция	Символ	SQL-аналог	Описание
Селекция	$\sigma_{\theta}(R)$	<code>WHERE</code>	Выбор кортежей, удовлетворяющих условию θ .
Проекция	$\pi_{A_1, \dots, A_k}(R)$	<code>SELECT col1, col2</code>	Выбор подмножества атрибутов.
Объединение	$R \cup S$	<code>UNION</code>	Кортежи, принадлежащие R или S .
Разность	$R - S$	<code>EXCEPT</code>	Кортежи из R , не

Операция	Символ	SQL-аналог	Описание
			принадлежащие S .
Декартово произведение	$R \times S$	CROSS JOIN	Все комбинации кортежей.
Переименование	$\rho_{S(A_1, \dots)}(R)$	AS	Новые имена отношению/атрибутам.
Соединение	$R \bowtie_{\theta} S$	JOIN ... ON	$\sigma_{\theta}(R \times S)$.

Почему это важно для практики:

SQL — **декларативный** язык. Вы описываете *что* хотите получить (реляционное выражение), а не *как* это получить. Оптимизатор запросов СУБД преобразует SQL в дерево операций реляционной алгебры и выбирает наиболее эффективный план выполнения (использование индексов, порядок соединений, методы агрегации).

Понимание реляционной алгебры помогает читать планы выполнения (`EXPLAIN ANALYZE`) и понимать, почему запрос медленный.

6.2. SQLAlchemy 2.0: асинхронный стиль

6.2.1. Эволюция: от Query API к Select API

SQLAlchemy 1.x доминировал десятилетие, но его API был выросшим органически, с несогласованностями между Core и ORM:

```
In [ ]: # SQLAlchemy 1.x – query-style
users = session.query(User).filter(User.age > 18).order_by(User.name).all()
```

SQLAlchemy 2.0 (2023) — переработка с нуля:

```
In [ ]: # SQLAlchemy 2.0 – select-style, unified API
result = await session.execute(select(User).where(User.age > 18).order_by(
users = result.scalars().all()
```

Ключевые изменения:

- Единый API для Core и ORM: `select()` , `insert()` , `update()` , `delete()` используются везде.
- Полная совместимость с `typing` — ORM-модели типизированы.

- Нативная поддержка асинхронности без «прослоек».

6.2.2. Асинхронное ядро: движок, сессии, контекстные менеджеры

Создание асинхронного движка

```
In [ ]: from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, asy

# Для PostgreSQL используем asyncpg
DATABASE_URL = "postgresql+asyncpg://user:pass@localhost/dbname"

engine = create_async_engine(
    DATABASE_URL,
    echo=True,           # логирование SQL (только для разработки)
    pool_size=10,        # базовое число соединений
    max_overflow=20,      # дополнительные соединения при пиках
    pool_recycle=3600,    # пересоздавать соединение каждый час
)
```

`create_async_engine` возвращает `AsyncEngine`, который оборачивает синхронный движок и делегирует I/O в event loop.

async_sessionmaker

```
In [ ]: AsyncSessionLocal = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False, # не сбрасывать объекты после commit
    autoflush=False,       # не делать flush перед каждым запросом
)
```

`expire_on_commit=False` критичен для асинхронного кода. По умолчанию SQLAlchemy «истекает» (expire) объекты после commit'a, и при следующем обращении к атрибуту делает ленивый запрос (`lazy load`). В асинхронном коде `lazy load` невозможен вне сессии — будет ошибка. `expire_on_commit=False` сохраняет данные в объекте.

Выполнение запросов

```
In [ ]: from sqlalchemy import select, insert, update, delete

async with AsyncSessionLocal() as session:
    # SELECT
    result = await session.execute(select(User).where(User.id == 1))
    user = result.scalar_one_or_none()
```



```

# INSERT
new_user = User(name="Alice", email="alice@example.com")
session.add(new_user)
await session.commit()

# UPDATE
await session.execute(
    update(User).where(User.id == 1).values(name="Bob")
)
await session.commit()

# DELETE
await session.execute(delete(User).where(User.id == 1))
await session.commit()

```

6.2.3. Эксплицитные транзакции

SQLAlchemy 2.0 поощряет **явное** управление транзакциями:

```

In [ ]: async with AsyncSessionLocal() as session:
        async with session.begin(): # <- начинает транзакцию
            session.add(user1)
            session.add(user2)
        # При выходе из контекста: commit при успехе, rollback при исключении

```

Эквивалентно:

```

In [ ]: async with AsyncSessionLocal() as session:
        try:
            session.add(user1)
            session.add(user2)
            await session.commit()
        except Exception:
            await session.rollback()
            raise
        finally:
            await session.close()

```

Почему explicit лучше implicit:

- `session.commit()` виден в коде — точка фиксации очевидна.
- Легче обрабатывать ошибки: `rollback` в `except`, `retry`-логика.
- Нет неожиданных `auto-commit`'ов.

6.2.4. Connection Pooling

Пул соединений — это кэш установленных TCP-соединений с БД.
Создание соединения дорого (TCP handshake + аутентификация)

PostgreSQL), поэтому пул переиспользует их.

Параметры `create_async_engine` :

Параметр	Значение	Смысл
<code>pool_size</code>	5	Сколько соединений держать постоянно открытыми.
<code>max_overflow</code>	10	Сколько дополнительных соединений создать при пике.
<code>pool_timeout</code>	30	Сколько секунд ждать свободного соединения из пула.
<code>pool_recycle</code>	-1	Через сколько секунд принудительно закрыть соединение (защита от «зомби»).
<code>pool_pre_ping</code>	False	Проверять соединение перед выдачей (дополнительный запрос, но защита от обрывов).

Как настроить под ML-сервис:

- Если среднее время запроса к БД — 10 мс, а RPS — 1000, то одно соединение обслуживает ~100 RPS.
- Нужно минимум $\$1000 / 100 = 10\$$ соединений. С запасом: `pool_size=20` , `max_overflow=10` .
- Но не слишком много: каждое соединение потребляет память в PostgreSQL (~10 МБ на рабочие буферы).

Пул как система массового обслуживания:

Пул соединений — это M/M/c/K система:

- $c = \text{pool_size} + \text{max_overflow}$ — число серверов (соединений).
- $K = c$ — очередь отсутствует (или бесконечна, если `pool_timeout > 0`).
- Если все c соединений заняты, новый запрос либо ждёт (`pool_timeout`), либо получает ошибку.

Вероятность отказа (все соединения заняты) по формуле Эрланга В:

$$P_b = \frac{\frac{\rho^c}{c!}}{\sum_{k=0}^c \frac{\rho^k}{k!}}$$
где $\rho = \frac{\lambda}{\mu}$ — загрузка на одно соединение. Если P_b высока — увеличивайте пул или оптимизируйте запросы.

6.3. Интеграция SQLAlchemy с FastAPI

6.3.1. Паттерн «сессия на запрос»

Глобальная сессия SQLAlchemy — антипаттерн в веб-приложениях.

Сессия хранит состояние (identity map, кэш объектов), разрастается и не потокобезопасна (в async — не корутинобезопасна).

Правильно: создавать сессию на каждый HTTP-запрос и закрывать после обработки.

```
In [ ]: from fastapi import Depends
        from sqlalchemy.ext.asyncio import AsyncSession

        async def get_db() -> AsyncSession:
            async with AsyncSessionLocal() as session:
                yield session
            # После yield: сессия автоматически закрывается
```

Использование в endpoint'e:

```
In [ ]: @app.get("/users/{user_id}")
        async def get_user(user_id: int, db: AsyncSession = Depends(get_db)):
            result = await db.execute(select(User).where(User.id == user_id))
            user = result.scalar_one_or_none()
            if not user:
                raise HTTPException(status_code=404, detail="User not found")
            return user
```

Что происходит под капотом:

1. FastAPI вызывает `get_db()`, получает генератор.
2. `async with AsyncSessionLocal()` создаёт сессию.
3. `yield session` передаёт сессию в `get_user`.
4. Endpoint выполняется, возможно делает `await db.commit()`.
5. После возврата ответа клиенту FastAPI возобновляет генератор.
6. Выполняется выход из `async with` — сессия закрывается.

6.3.2. Транзакции через Depends и Unit of Work

Расширим паттерн до полноценного Unit of Work:

```
In [ ]: class DatabaseUnitOfWork:
        def __init__(self, session: AsyncSession):
            self.session = session

        async def commit(self):
            await self.session.commit()
```

```

    async def rollback(self):
        await self.session.rollback()

async def get_uow(session: AsyncSession = Depends(get_db)) -> DatabaseUnitOfWork:
    uow = DatabaseUnitOfWork(session)
    try:
        yield uow
        await uow.commit()
    except Exception:
        await uow.rollback()
        raise

```

Репозиторий, использующий UoW:

```

In [ ]: class UserRepository:
        def __init__(self, session: AsyncSession):
            self._session = session

        async def get_by_id(self, user_id: int) -> User | None:
            result = await self._session.execute(select(User).where(User.id == user_id))
            return result.scalar_one_or_none()

        async def create(self, user: User) -> User:
            self._session.add(user)
            await self._session.flush() # отправляет в БД, но не коммитит
            return user

    async def get_user_repo(session: AsyncSession = Depends(get_db)) -> UserRepository:
        return UserRepository(session)

@app.post("/users/")
async def create_user(
    user_data: UserCreate,
    repo: UserRepository = Depends(get_user_repo),
    uow: DatabaseUnitOfWork = Depends(get_uow),
):
    user = User(**user_data.model_dump())
    await repo.create(user)
    # commit произойдёт автоматически при успешном выходе из get_uow
    return user

```

Важный нюанс: `get_db` кэшируется (`use_cache=True` по умолчанию). Поэтому `repo` и `uow` получают **одну и ту же** сессию. Это гарантирует, что `repo.create()` и `uow.commit()` работают в рамках одной транзакции.

6.3.3. Обработка deadlock'ов и retry-логика

Что такое deadlock?

Deadlock возникает, когда две транзакции блокируют ресурсы перекрёстно:

```
In [ ]: Транзакция A: UPDATE accounts WHERE id = 1 (блокирует строку 1)
        Транзакция B: UPDATE accounts WHERE id = 2 (блокирует строку 2)

        Транзакция A: UPDATE accounts WHERE id = 2 -> ждёт B
        Транзакция B: UPDATE accounts WHERE id = 1 -> ждёт A

        DEADLOCK! PostgreSQL убивает одну из транзакций.
```

PostgreSQL автоматически обнаруживает deadlock (параметр `deadlock_timeout`, по умолчанию 1 секунда) и прерывает одну из транзакций с ошибкой:

```
In [ ]: asyncpg.exceptions.DeadlockDetectedError: deadlock detected
```

Retry-логика на уровне приложения

Deadlock — это **ожидаемая** ситуация при высокой конкурентности. Приложение должно перезапускать транзакцию:

```
In [ ]: import random
        from tenacity import retry, stop_after_attempt, wait_exponential_jitter

        class DatabaseError(Exception):
            pass

        @retry(
            stop=stop_after_attempt(3),
            wait=wait_exponential_jitter(initial=0.1, max=1),
            retry=lambda e: isinstance(e, DatabaseError),
        )
        async def transfer_funds(from_id: int, to_id: int, amount: float, db: AsyncpgDB):
            try:
                async with db.begin():
                    from_acc = await db.get(Account, from_id)
                    to_acc = await db.get(Account, to_id)

                    if from_acc.balance < amount:
                        raise ValueError("Insufficient funds")

                    from_acc.balance -= amount
                    to_acc.balance += amount
            except DeadlockDetectedError:
                raise DatabaseError("Deadlock, retrying...")
```

Почему jitter? При массовом deadlock'е (например, все клиенты пытаются обновить одни и те же строки) синхронный retry приведёт к **thundering herd**: все транзакции повторят попытку одновременно и снова заблокируют друг друга. Случайный jitter размазывает повторные попытки во времени.

Оптимистичные блокировки

Вместо пессимистических блокировок (`SELECT FOR UPDATE`) можно использовать версионирование:

```
In [ ]: class Account(Base):
    __tablename__ = "accounts"

    id: Mapped[int] = mapped_column(primary_key=True)
    balance: Mapped[float]
    version: Mapped[int] = mapped_column(default=0) # счётчик версий

    async def transfer_optimistic(from_id: int, to_id: int, amount: float, db):
        while True:
            from_acc = await db.get(Account, from_id)
            to_acc = await db.get(Account, to_id)

            from_acc.balance -= amount
            to_acc.balance += amount
            from_acc.version += 1
            to_acc.version += 1

            try:
                await db.commit()
                break
            except IntegrityError: # версия изменилась другой транзакцией
                await db.rollback()
                continue # повторяем с начала
```

Проверка версии обычно делается через `WHERE id = ? AND version = ?`, что автоматически вызывает ошибку, если строка была изменена.

6.4. Миграции: Alembic

6.4.1. Зачем нужны миграции

Код эволюционирует: добавляются поля, меняются типы, создаются индексы. База данных должна соответствовать коду. Ручное изменение схемы (`ALTER TABLE` вручную) — путь к хаосу в команде.

Миграция — это скрипт, описывающий переход схемы БД от версии N к версии $N+1$ (и обратно).

6.4.2. Инициализация и работа с Alembic

```
In [ ]: # Установка
pip install alembic

# Инициализация
alembic init alembic
```

Создаётся структура:

- `alembic.ini` — конфигурация (URL БД, путь к скриптам).
- `alembic/versions/` — директория с файлами миграций.
- `alembic/env.py` — скрипт окружения, где подключаются модели SQLAlchemy.

Настройка `env.py` для `async`:

```
In [ ]: # alembic/env.py
from sqlalchemy.ext.asyncio import create_async_engine
from myapp.models import Base # ваши модели

config.set_main_option("sqlalchemy.url", "postgresql+asyncpg://...")

def run_migrations_online():
    connectable = create_async_engine(config.get_main_option("sqlalchemy.

    with connectable.connect() as connection:
        context.configure(connection=connection, target_metadata=Base.met
        with context.begin_transaction():
            context.run_migrations()
```

Создание миграции:

```
In [ ]: alembic revision --autogenerate -m "Add users table"
```

Alembic сравнивает текущую схему БД с метаданными SQLAlchemy (`Base.metadata`) и генерирует скрипт:

```
In [ ]: """Add users table

Revision ID: a1b2c3d4
Revises:
Create Date: 2024-01-15 10:00:00.000000
```

```

"""
from alembic import op
import sqlalchemy as sa

# revision identifiers, used by Alembic.
revision = 'a1b2c3d4'
down_revision = None
branch_labels = None
depends_on = None

def upgrade():
    op.create_table(
        'users',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('name', sa.String(), nullable=False),
        sa.Column('email', sa.String(), nullable=False),
        sa.PrimaryKeyConstraint('id'),
        sa.UniqueConstraint('email')
    )

def downgrade():
    op.drop_table('users')

```

Применение миграции:

```

In [ ]: alembic upgrade head      # до последней версии
        alembic upgrade +1      # на одну версию вперёд
        alembic downgrade -1    # на одну версию назад
        alembic downgrade base  # до начального состояния

```

6.4.3. Проверка корректности и zero-downtime миграции

Autogenerate не идеален. Он не умеет:

- Распознавать некоторые изменения типов (например, `VARCHAR(50)` -> `VARCHAR(100)` в PostgreSQL — безопасно, но Alembic может сгенерировать ненужную миграцию).
- Переименовывать колонки (видит как `drop` + `add`).
- Создавать сложные миграции данных.

Всегда проверяйте сгенерированный скрипт вручную.

Стратегия Expand/Contract

Для zero-downtime деплоя (без простоя сервиса) используется подход:

1. **Expand (расширение):** добавить новую колонку/таблицу, не удаляя

- старую. Код пишет в обе, читает из старой.
2. **Migrate (миграция данных):** перенести данные из старой структуры в новую.
 3. **Contract (сжатие):** переключить код на чтение из новой структуры. Удалить старую.

Пример: переименование колонки `username` в `login`.

```
In [ ]: War 1 (Expand):
        ALTER TABLE users ADD COLUMN login VARCHAR(255);
        # Код пишет в username и login

War 2 (Migrate):
        UPDATE users SET login = username WHERE login IS NULL;

War 3 (Contract):
        # Код читает из login
        ALTER TABLE users DROP COLUMN username;
```

6.4.4. Миграции в CI/CD

```
In [ ]: # .github/workflows/deploy.yml
jobs:
  migrate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run migrations
        run: |
          pip install alembic asyncpg
          alembic upgrade head
        env:
          DATABASE_URL: ${ secrets.DATABASE_URL }

      - name: Deploy app
        run: |
          # деплой только после успешных миграций
```

Правило: миграции запускаются **до** деплоя новой версии приложения. Иначе новый код обратится к старой схеме и упадёт.

6.5. NoSQL и специализированные хранилища

6.5.1. MongoDB с Motor

MongoDB — документо-ориентированная БД. Полезна, когда:

- схема данных гибкая (логи, метаданные ML-экспериментов);
- нужна горизонтальная масштабируемость (шардирование из коробки);
- запросы простые, без сложных JOIN.

Motor — официальный асинхронный драйвер для Python.

```
In [ ]: from motor.motor_asyncio import AsyncIOMotorClient

client = AsyncIOMotorClient("mongodb://localhost:27017")
db = client.ml_logs

@app.post("/log-prediction")
async def log_prediction(prediction: PredictionLog):
    result = await db.predictions.insert_one(prediction.model_dump())
    return {"id": str(result.inserted_id)}

@app.get("/predictions")
async def get_predictions(model_version: str):
    cursor = db.predictions.find({"model_version": model_version})
    predictions = await cursor.to_list(length=100)
    return predictions
```

6.5.2. Redis: кэш, pub/sub, rate limiting

Redis — хранилище ключ-значение в памяти. Идеально для:

- **Кэширования:** хранение часто запрашиваемых данных (результаты ML-предсказаний для одинаковых входов).
- **Pub/Sub:** real-time уведомления (например, о завершении обучения модели).
- **Rate Limiting:** ограничение числа запросов от одного клиента.
- **Очередей:** списки и стримы для фоновых задач.

```
In [ ]: import redis.asyncio as redis

redis_client = redis.Redis(host='localhost', port=6379, decode_responses=

# Кэширование с TTL
@app.get("/predict/{input_hash}")
async def cached_predict(input_hash: str):
    cached = await redis_client.get(f"pred:{input_hash}")
    if cached:
        return json.loads(cached)
```

```

    result = await expensive_ml_predict(input_hash)
    await redis_client.setex(f"pred:{input_hash}", 3600, json.dumps(result))
    return result

# Pub/Sub
@app.post("/notify")
async def notify(channel: str, message: str):
    await redis_client.publish(channel, message)
    return {"status": "published"}

# Rate Limiting (sliding window)
async def rate_limit_check(client_id: str, limit: int = 100, window: int):
    key = f"rate_limit:{client_id}"
    current = await redis_client.get(key)
    if current and int(current) >= limit:
        raise HTTPException(status_code=429, detail="Too many requests")

    pipe = redis_client.pipeline()
    pipe.incr(key)
    pipe.expire(key, window)
    await pipe.execute()

```

6.5.3. Векторные БД для ML

Традиционные БД плохо справляются с **поиском по сходству** (similarity search) в пространствах высокой размерности. Векторные базы данных спроектированы для хранения **embeddings** — плотных числовых векторов, представляющих семантику текста, изображений, аудио.

Задача: найти k ближайших соседей к запросному вектору q в $\mathbb{R}^d$ по метрике ρ .

Метрики:

- **Cosine similarity:** $\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$ — для семантического поиска (направление важнее длины).
- **Euclidean distance:** $\|u - v\|_2$ — для кластеризации.
- **Dot product:** $u \cdot v$ — для рекомендательных систем.

Алгоритмы индексирования:

- **Flat (brute-force):** точный поиск, $O(n \cdot d)$. Медленно для $n > 10^6$.
- **HNSW (Hierarchical Navigable Small World):** графовый приближённый индекс. Баланс между скоростью и точностью.
- **IVF (Inverted File Index):** кластеризация векторов, поиск только в ближайших центроидах.

Сравнение векторных БД:

База	Тип	Особенности
Pinecone	Managed SaaS	Простота, масштабирование без усилий, дорого.
Weaviate	Open-source / Cloud	Гибридный поиск (векторный + ключевые слова), GraphQL.
Qdrant	Open-source / Cloud	Rust, фильтрация метаданных, распределённость.
Chroma	Open-source	Локальная, встраиваемая, для прототипов.
pgvector	Extension PostgreSQL	Хранение векторов в привычной PostgreSQL, HNSW-индексы.

Пример с Qdrant:

```
In [ ]: from qdrant_client import AsyncQdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct

client = AsyncQdrantClient(host="localhost", port=6333)

# Создание коллекции
await client.create_collection(
    collection_name="text_embeddings",
    vectors_config=VectorParams(size=768, distance=Distance.COSINE)
)

# Добавление векторов
points = [
    PointStruct(id=1, vector=[0.1, 0.2, ...], payload={"text": "hello"}),
    PointStruct(id=2, vector=[0.3, 0.4, ...], payload={"text": "world"}),
]
await client.upsert(collection_name="text_embeddings", points=points)

# Поиск
results = await client.search(
    collection_name="text_embeddings",
    query_vector=[0.1, 0.25, ...],
    limit=3
)
```

Когда использовать векторную БД в ML-сервисе:

- Semantic search: поиск документов по смыслу, а не по ключевым словам.
- RAG (Retrieval-Augmented Generation): извлечение релевантных контекстов для LLM.
- Дедупликация: поиск похожих изображений/текстов.
- Рекомендации: «пользователи с похожими embeddings».

Итог модуля 6

Концепция	Суть	Практическое применение
ACID	Атомарность, согласованность, изоляция, долговечность	Гарантия целостности данных ML-сервиса
Уровни изоляции	Компромисс между корректностью и скоростью	READ COMMITTED для API, SERIALIZABLE для финансов
Реляционная алгебра	Математическая основа SQL	Понимание планов выполнения запросов
B-Tree	Сбалансированное дерево для индексов	$O(\log n)$ поиск, критичен для WHERE user_id = ?
SQLAlchemy 2.0	Единый async/sync API, select-style	Современный ORM для FastAPI
Connection Pool	Кэш TCP-соединений с БД	Настройка pool_size под нагрузку
Сессия на запрос	get_db() с yield и AsyncSession	Изоляция транзакций, автоматическое закрытие
Deadlock	Циклическое ожидание блокировок	Retry с jitter, оптимистичные блокировки
Alembic	Версионирование схемы БД	Командная разработка, CI/CD
Expand/Contract	Zero-downtime миграции	Безостановочный деплой
Redis	In-memory хранилище	Кэш предсказаний, rate limiting, pub/sub
Векторные БД	Хранение embeddings, similarity search	RAG, семантический поиск, рекомендации

В **Модуле 7** мы перейдём к безопасности: аутентификация, авторизация, криптография. Разберём, как защитить ML-API от несанкционированного доступа и злоупотреблений.